Full Stack Intern – Take-Home Project

> **Project:** EventBook, a mini event booking app
> **Confirmation of participation:** please reply by Friday, 16 October 2026, 6:00 PM NPT
> **Submission deadline:** Sunday, 18 October 2026, 6:00 PM NPT
> **Expected effort:** about 10–14 hours. The scope is deliberately small. We care about **correct, clean and deployed**, not big.

1. The idea

An **admin** publishes events (a workshop, a meetup, a concert) with a fixed number of seats. **Visitors** can browse events without logging in. **Registered users** can book a seat and cancel it later. Seats must never be overbooked.

2. Tech stack

Layer	Option A – Node.js (preferred)	Option B – Python backend
Frontend	**Next.js + TypeScript**, required for both options. Any styling approach.	
Backend	**Node.js + Express**	**Django REST Framework** or **FastAPI**
Database	**Supabase** (PostgreSQL), required for both options	

The frontend and backend must be **separate**: your Next.js app talks to **your** REST API, and only your backend talks to Supabase. Use Next.js for the frontend only; no Next.js API routes or server actions as the backend, and no Supabase queries from Next.js. Never expose the Supabase service-role key in frontend code. Build login and registration in your own backend instead of using Supabase Auth; we want to see how you handle it.

3. What the app must do

Visitors (not logged in)

- Browse **upcoming** events and see how many **seats are left** on each.
- **Search** events by title.
- The list is **paginated on the server**, not by loading everything and slicing on the frontend.
- Open an event to see its full details.

Users

- **Register, log in and log out.**
- **Book a seat** for an event (one seat per booking).
- See **their own bookings** and **cancel** one.

Admin

- **Create, edit and delete** events. An event has at least a title, description, date and time, venue and total seats.
- See **who has booked** a given event.
- Create the admin account through a seed script. Nobody should be able to make themselves admin through the app.

4. Rules your backend must enforce

These must hold even if someone skips your UI and calls the API directly with Postman or curl.

1. A user **cannot book the same event twice**, unless they cancelled their earlier booking.
2. An event **cannot be overbooked**.
3. Past events **cannot be booked**, and bookings for them cannot be cancelled.
4. Users can only cancel **their own** bookings. Cancelling frees the seat.
5. An admin cannot reduce an event's seats **below the number already booked**.
6. Only admins can manage events or see attendee lists.

Think about what *should* happen in edge cases the list doesn't cover, such as deleting an event that people have booked. Decide, implement it and explain your choice in the README.

5. You decide the design

We intentionally **don't** give you a database schema, API routes or page layouts. Designing them is part of the test. You choose:

- your **data model** and how you calculate seats left
- your **API design**: routes, request and response shapes, error format
- your **pages, navigation and UI**
- your **folder structure** and libraries

Keep it simple and consistent, and explain the important decisions in your README. We judge the reasoning, not whether it matches some "correct" answer.

6. Quality we expect

- **Security:** passwords hashed; protected routes really protected; secrets only in environment variables (**never commit** `.env`; commit a `.env.example`).
- **Validation and errors:** validate input on the server; return sensible HTTP status codes and clear error messages; never leak stack traces.
- **Frontend:** show loading, error and empty states (no blank screens, no silent failures); clear feedback when an action fails, for example "event is full"; usable on a phone.
- **Seed data:** one admin, one normal user and enough sample events to test pagination and the rules (include a past event and a nearly full one).
- **Tests:** at least **3 automated backend tests** for your most important rules.
- **Git:** commit as you go, with meaningful messages. A history of 1–3 giant commits is a red flag.

7. Deployment (COMPULSORY)

Your app must be **live on the internet**. Submissions without a working live link will not be reviewed.

- Use any host. Free tiers are enough, for example Vercel or Netlify for the frontend, Render or Railway for the backend, and a free Supabase project for the database.
- Seed the live database so we can log in immediately with the test accounts in your README.
- Keep it live for at least **2 weeks** after submission. If a free backend sleeps (first request is slow), mention it in the README. That's fine.

8. AI tools: allowed, but no "AI slop"

You may use ChatGPT, Claude, Copilot, Cursor and similar tools. We use them too. But you are the developer, not the AI. Make sure you understand what you build. If you're shortlisted, we'll have a short, friendly chat where you walk us through your project.

What we consider AI slop (it costs you points):

- Code you cannot explain when asked.
- Unused files, dead code, commented-out blocks or leftover boilerplate.
- Inconsistent patterns that show code pasted from different chats.
- A long, generic README that doesn't match what your code actually does.
- Features that look present in the UI but don't really work, or rules enforced only in the frontend.
- Over-engineering a small app.

Tell us honestly in the README **which AI tools you used and for what**.

9. What to submit

Before the deadline (**Sunday, 18 October 2026, 6:00 PM NPT**), reply to this email with:

1. Your **live application URL** (and API URL)
2. Your **GitHub repository link** (public, or invite us)
3. A **README**, in your own words, covering:

```
Live links & test accounts     admin + user logins
Tech stack                     and why
Run locally                    steps, seed command, test command
Data model                     your tables and why
API                            your endpoints, briefly
Decisions & trade-offs         e.g. seat counting, edge cases you handled
AI usage                       tools used and for which parts
Known issues / next steps
```

Couldn't finish everything? Submit anyway.

We value **effort, honesty and how you think**, not just a completed checklist. If time runs out, submit what you have by the deadline. Deploy the parts that work, and use the **"Known issues / next steps"** section of your README to tell us what's missing and how you would build it.

A smaller app that works and is honestly explained is better than a bigger one that's broken. If you're short on time, prioritise in this order: **login** → **booking rules** → **admin event management** → **frontend polish**.

10. Suggested plan

- **Day 1:** design the data model, build auth and the backend with all rules and the seed script. Check everything with Postman.
- **Day 2:** Next.js + TypeScript frontend, connected to your API. Deploy a first version **early**.
- **Day 3:** tests, fixes and polish, then the final deployment and README. Leave buffer time; deployment always takes longer than you think.

11. Bonus (only after everything above works)

- Handle **two users booking the last seat at the same moment** safely, and explain how.
- Docker Compose for local development.
- Rate limiting on login.

12. Self-check before you submit

- ☐ Register, login, booking and cancelling all work on the **live** site.
- ☐ Booking twice, booking a full event and booking a past event all fail with a clear message.
- ☐ Calling your admin APIs as a normal user (via Postman) is rejected.
- ☐ Search and pagination work on the live site.
- ☐ `.env` is not in the repo, and the tests pass with one command.
- ☐ I can explain every file in my repo.

How we evaluate

1. **Works correctly on the live site:** features and rules (biggest weight)
2. **Design and code quality:** sensible data model and API, clean consistent code, no slop
3. **Security and validation**
4. **Frontend UX**
5. **README, Git history and tests**

A final note

Whether or not you get selected, this project is worth your time. In three days you'll work with everything a real product needs: authentication, business rules, a database, an API, a frontend and a live deployment. That is real experience.

So don't build it just to pass a screening. Build something **you're proud of**, something you'd happily pin on your GitHub and show in your portfolio. Learn, experiment and enjoy the process.

All the best. We're excited to see what you build!